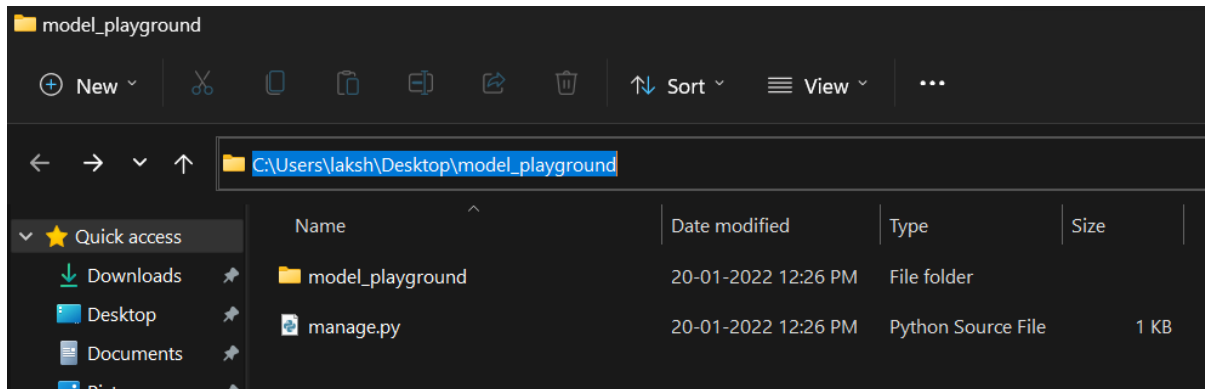


- We create a new project in Django names **model\_playground** at the desktop.

```
(base) C:\Users\laksh\Desktop>django-admin startproject model_playground
```



Now **model\_playground** contain one folder with same name and also a **manage.py** file which is a main file that needs to be run to deploy are project on web.

- Now we create virtual environment inside the **model\_ground** folder.

```
(base) C:\Users\laksh\Desktop>cd model_playground
(base) C:\Users\laksh\Desktop\model_playground>virtualenv env
created virtual environment (CPython3.8.8.final.0-64 in 1954ms)
creator CPython3Windows(dest=C:\Users\laksh\Desktop\model_playground\env, clear=False, no_vcs_ignore=False, global=False)
seeder FromAppData(download=False, pip=bundle, setuptools=bundle, wheel=bundle, via=copy, app_data_dir=C:\Users\laksh\AppData\Local\pypa\virtualenv)
added seed packages: pip==21.3.1, setuptools==60.2.0, wheel==0.37.1
activators BashActivator,BatchActivator,FishActivator,NushellActivator,PowerShellActivator,PythonActivator
```

Virtual environment is created with a name **env**.

```
(base) C:\Users\laksh\Desktop\model_playground>env\Scripts\activate
(env) (base) C:\Users\laksh\Desktop\model_playground>
```

Activated the env and now install the required python module in this virtual environment using pip install command.

```
(env) (base) C:\Users\laksh\Desktop\model_playground>pip install django
Collecting django
Using cached Django-4.0.1-py3-none-any.whl (8.0 MB)
```

- Now we create a app which is say equivalent to major webpage of our website.

```
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py startapp main

(env) (base) C:\Users\laksh\Desktop\model_playground>ls -l
total 9
drwxr-xr-x 4 laksh Administrators    0 Jan 20 12:33 env
drwxr-xr-x 3 laksh Administrators 4096 Jan 20 12:37 main
-rwxr-xr-x 1 laksh Administrators  694 Jan 20 12:26 manage.py
drwxr-xr-x 3 laksh Administrators 4096 Jan 20 12:37 model_playground
```

See that main folder is get created using **startapp** command.

- To run the server

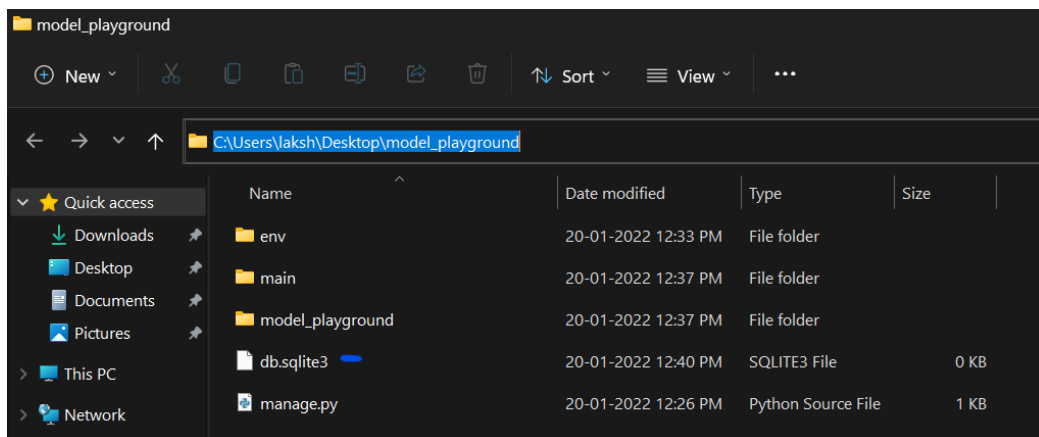
```
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

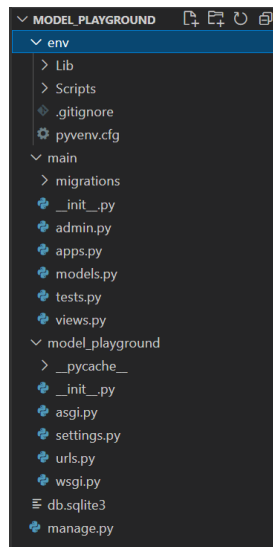
You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
January 20, 2022 - 12:40:54
Django version 4.0.1, using settings 'model_playground.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Now the webpage will be running at the <http://127.0.0.1:8000/>.

And when we run the manage.py for the first time it create a **db.sqlite3** file which is basically a sql database.



- These are the files that are created:



- If we check now the tables present in the db.sqlite3 it shows nothing.

```
(env) (base) C:\Users\laksh\Desktop\model_playground>sqlite3 db.sqlite3
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sqlite> .tables
sqlite>
```

After running a command: **python manage.py migrate** it will create the tables in the db.sqlite3 database.

```
(env) (base) C:\Users\laksh\Desktop\model_playground>sqlite3 db.sqlite3
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sqlite> .tables
sqlite> .quit

(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK

(env) (base) C:\Users\laksh\Desktop\model_playground>sqlite3 db.sqlite3
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sqlite> .tables
auth_group                  auth_user_user_permissions
auth_group_permissions      django_admin_log
auth_permission             django_content_type
auth_user                   django_migrations
auth_user_groups            django_session
sqlite>
```

- Now we can create our table by writing a class in **models.py** file of our created app using startapp command above which is being named as **main** in our case.

The screenshot shows the Visual Studio Code interface with the Django project 'model\_playground' open. The Explorer sidebar on the left shows the project structure, with 'model\_playground' selected. The main editor displays the content of 'models.py', which includes imports for Django and a 'Student' model with fields for name, roll\_no, course, and branch. The file explorer on the left shows the project structure with 'model\_playground' selected. The main editor displays the content of models.py, which includes imports for Django and a 'Student' model with fields for name, roll\_no, course, and branch.

Then add our app in **settings.py** file of **model\_playground** folder.

File Edit Selection View Go Run Terminal Help settings.py - model\_playground - Visual Studio Code

EXPLORER ... settings.py X

OPEN EDITORS

X settings.py model\_playground

MODEL\_PLAYGROUND

env

main

model\_playground

\_\_pycache\_\_

\_\_init\_\_.py

asgi.py

settings.py

urls.py

wsgi.py

db.sqlite3

manage.py

```

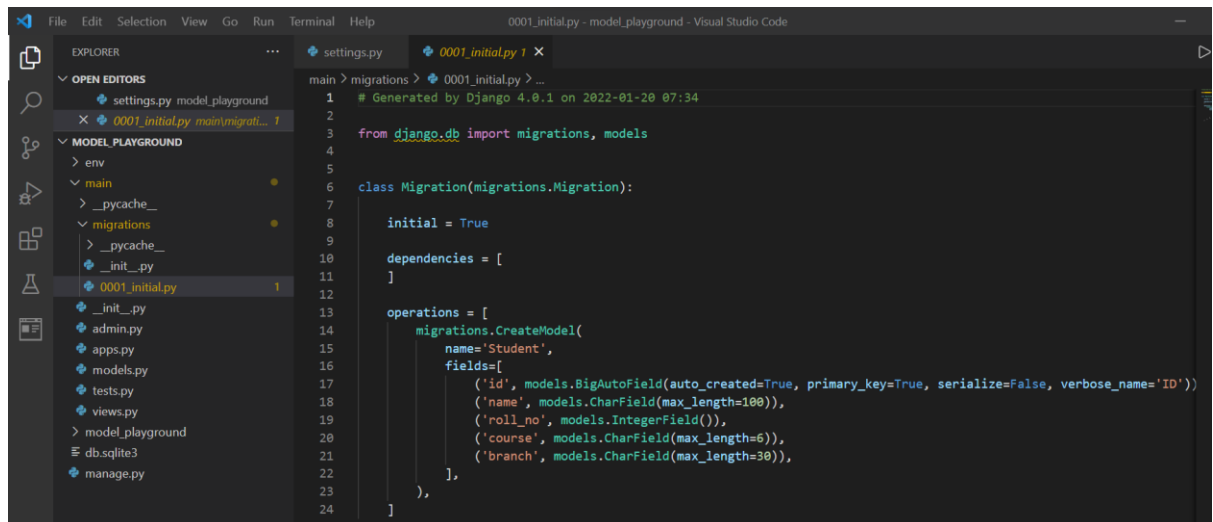
model_playground > settings.py > ...
25 SECRET_KEY = 'django-insecure-13mav70ajimud0ic-00qwas0p7mimj2bwitds-mlq...'
26
27 # SECURITY WARNING: don't run with debug turned on in production!
28 DEBUG = True
29
30 ALLOWED_HOSTS = []
31
32 # Application definition
33
34 INSTALLED_APPS = [
35     'main',
36     'django.contrib.admin',
37     'django.contrib.auth',
38     'django.contrib.contenttypes',
39     'django.contrib.sessions',
40     'django.contrib.messages',
41     'django.contrib.staticfiles',
42 ]

```

Now to populate the table in **db.sqlite3** file do the following:

```
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py makemigrations
Migrations for 'main':
  main\migrations\0001_initial.py
    - Create model Student
```

Now, it will create a file inside `main/migrations/` that have command to create a table with attributes you had declared in the `models.py` file.



Now to migrate the table in db.sqlite3 file:

```
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, main, sessions
Running migrations:
  Applying main.0001 initial... OK

(env) (base) C:\Users\laksh\Desktop\model_playground>sqlite3 db.sqlite3
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sqlite> .tables
auth_group          django_admin_log
auth_group_permissions  django_content_type
auth_permission      django_migrations
auth_user            django_session
auth_user_groups     main_student
auth_user_user_permissions
sqlite>
```

See that the table is created with a name **main\_Student**, where **main** is our (app name) and **Student** is the class name with which we created a class in **models.py** file.

- Now we access the default admin panel of the Django to see the data in our table. The steps are to be followed:

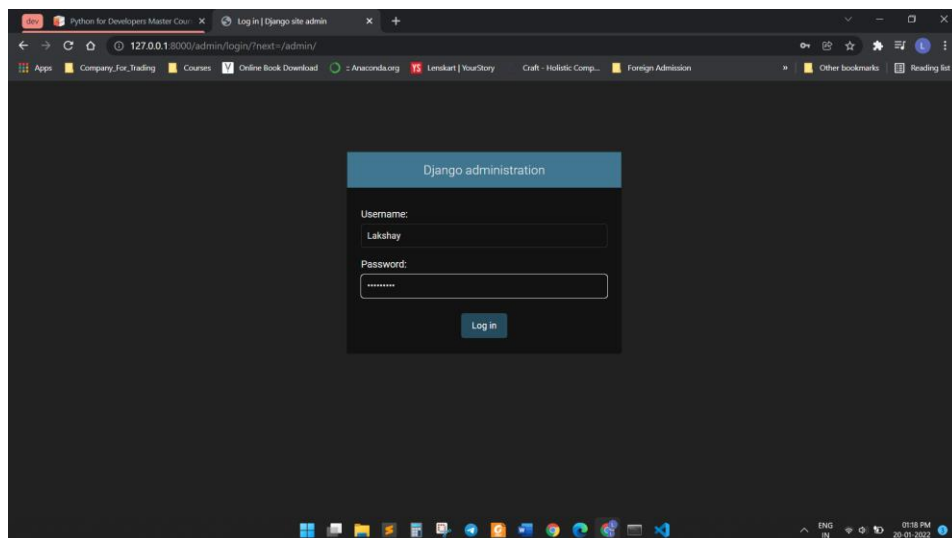
```
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py createsuperuser
Username (leave blank to use 'laksh'): Lakshay
Email address:
Password:
Password (again):
Superuser created successfully.

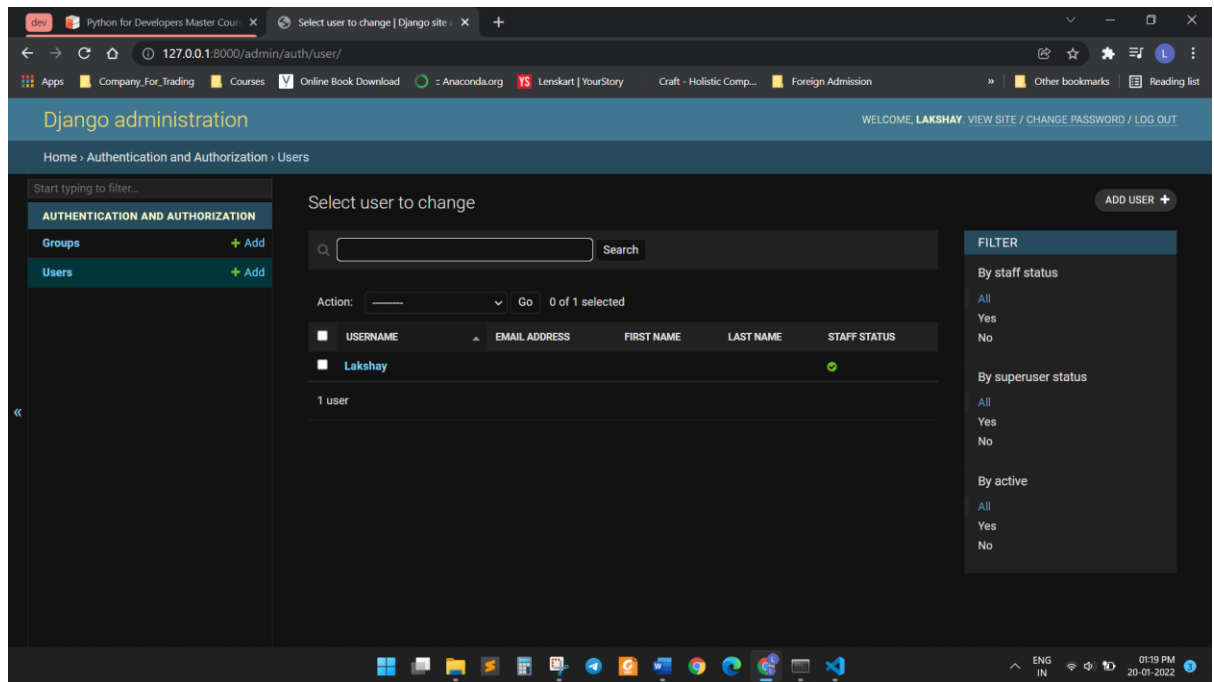
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
January 20, 2022 - 13:15:13
Django version 4.0.1, using settings 'model_playground.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

We create the login credential for the admin using **createsuperuser** command. The password that we enter is not visible on the screen.

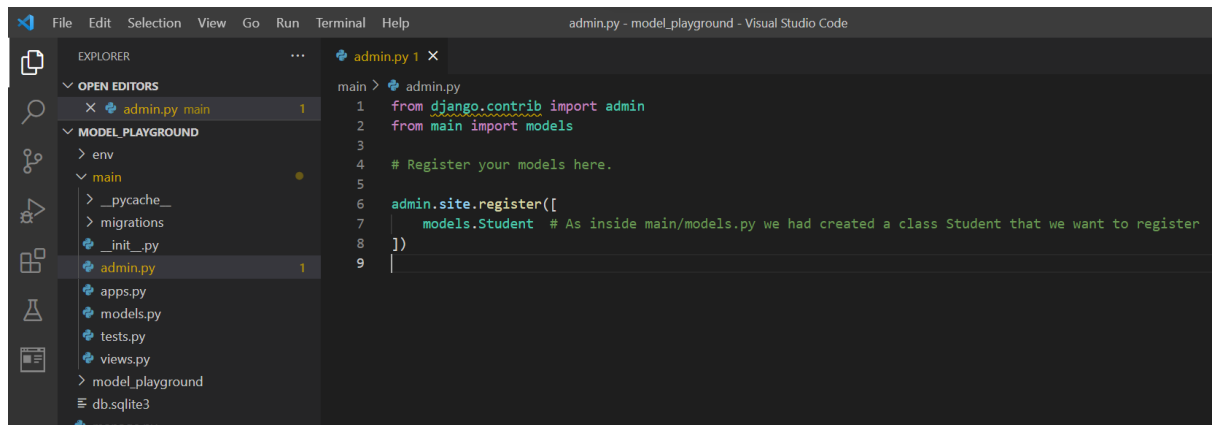
After running the server, now we can login in the admin page.



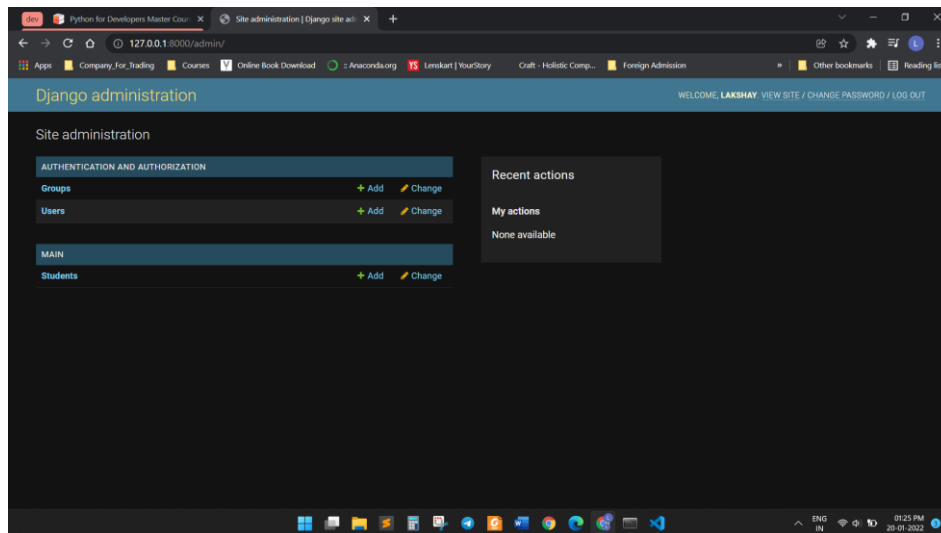


See inside user it shows the login credential of the user that are register on the website.

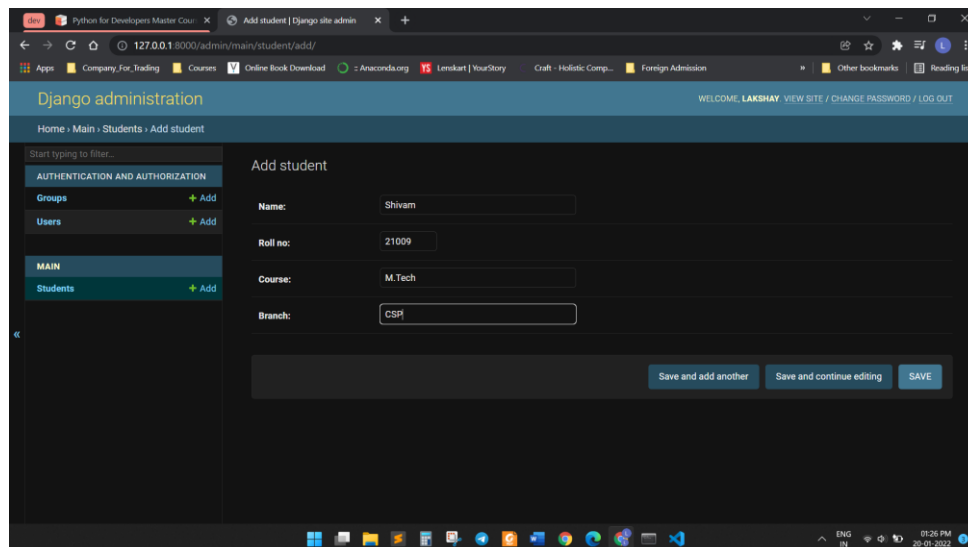
To populate our table on this page we need to do the following things:



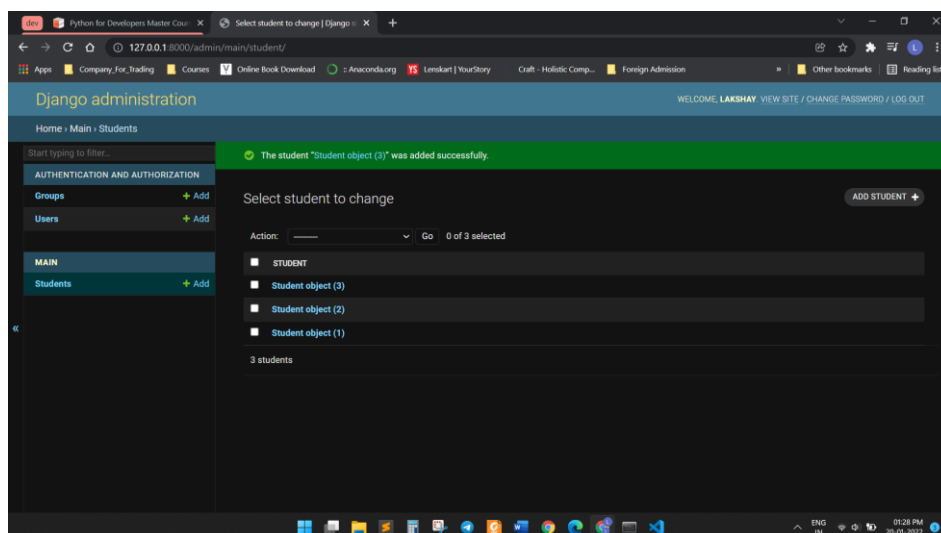
We need to register our table in the **admin.py** file inside our main folder. Now if we reload the webpage our table will be there.



Now see on home page of admin our Student table is present. We can enter the students details using this website.



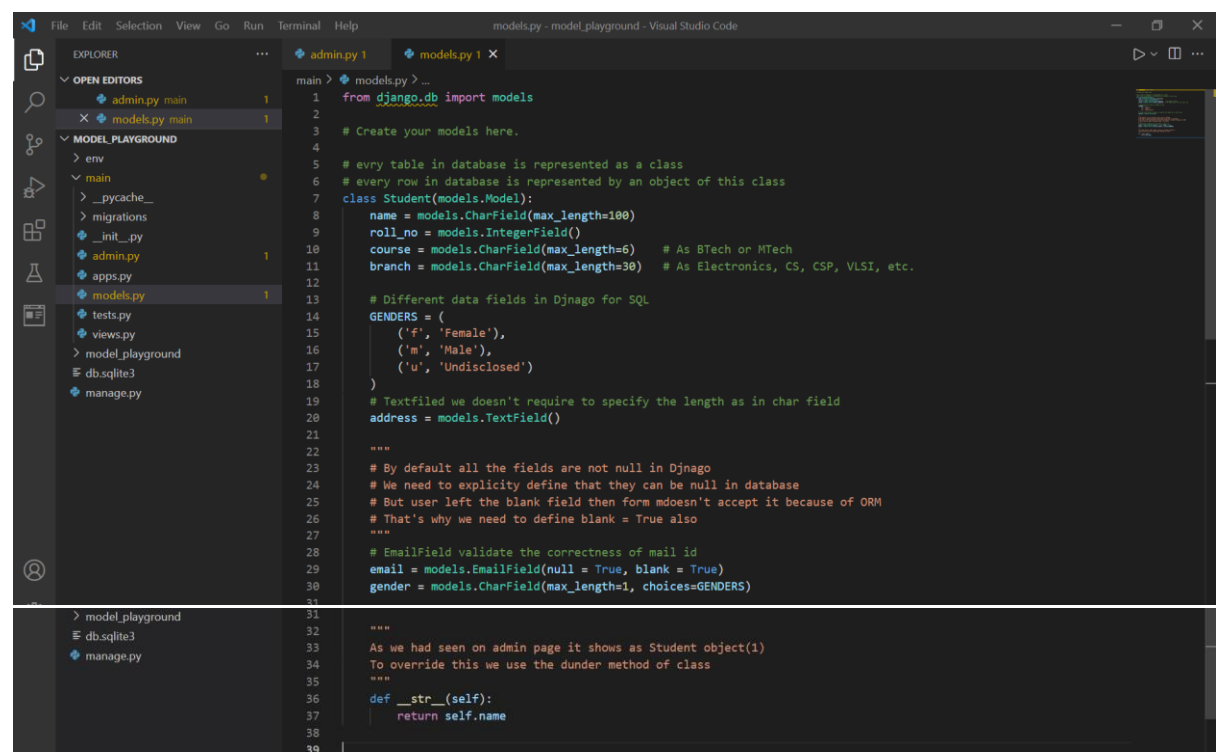
Similarly, we can add as many student we want to the table Student. Below shown is that we had added records of three student.



Now to see the data from backend by:

```
(env) (base) C:\Users\laksh\Desktop\model_playground>sqlite3 db.sqlite3
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sqlite> .tables
auth_group          django_admin_log
auth_group_permissions  django_content_type
auth_permission     django_migrations
auth_user           django_session
auth_user_groups    main_student
auth_user_user_permissions
sqlite> select * from main_student
...>
1|Shivam|21009|M.Tech|CSP
2|Shubham|21008|M.Tech|CSP
3|Manish|20003|B.Tech|Mechanical
sqlite>
```

- Some more fields and functionality in Django models.py



```
models.py - model_playground - Visual Studio Code
main > models.py > ...
1 from django.db import models
2
3 # Create your models here.
4
5 # evry table in database is represented as a class
6 # every row in database is represented by an object of this class
7 class Student(models.Model):
8     name = models.CharField(max_length=100)
9     roll_no = models.IntegerField()
10    course = models.CharField(max_length=6) # As BTech or MTech
11    branch = models.CharField(max_length=30) # As Electronics, CS, CSP, VLSI, etc.
12
13    # Different data fields in Djnago for SQL
14    GENDERS = (
15        ('f', 'Female'),
16        ('m', 'Male'),
17        ('u', 'Undisclosed')
18    )
19    # Textfiled we doesn't require to specify the length as in char field
20    address = models.TextField()
21
22    """
23    # By default all the fields are not null in Djnago
24    # We need to explicitly define that they can be null in database
25    # But user left the blank field then form mdoesn't accept it because of ORM
26    # That's why we need to define blank = True also
27    """
28    # EmailField validate the correctness of mail id
29    email = models.EmailField(null = True, blank = True)
30    gender = models.CharField(max_length=1, choices=GENDERS)
31
32    """
33    As we had seen on admin page it shows as Student object(1)
34    To override this we use the dunder method of class
35    """
36    def __str__(self):
37        return self.name
38
39
```

Code:

```
from django.db import models

# Create your models here.

# evry table in database is represented as a class
# every row in database is represented by an object of this class
class Student(models.Model):
    name = models.CharField(max_length=100)
    roll_no = models.IntegerField()
    course = models.CharField(max_length=6) # As BTech or MTech
```

```

        branch = models.CharField(max_length=30)    # As Electronics, CS, CSP,
VLSI, etc.

# Different data fields in Django for SQL
GENDERS = (
    ('f', 'Female'),
    ('m', 'Male'),
    ('u', 'Undisclosed')
)
# TextField we doesn't require to specify the length as in char field
address = models.TextField()

"""
# By default all the fields are not null in Django
# We need to explicitly define that they can be null in database
# But user left the blank field then form doesn't accept it because of
ORM
# That's why we need to define blank = True also
"""

# EmailField validate the correctness of mail id
email = models.EmailField(null = True, blank = True)
gender = models.CharField(max_length=1, choices=GENDERS)

"""
As we had seen on admin page it shows as Student object(1)
To override this we use the dunder method of class
"""
def __str__(self):
    return self.name

```

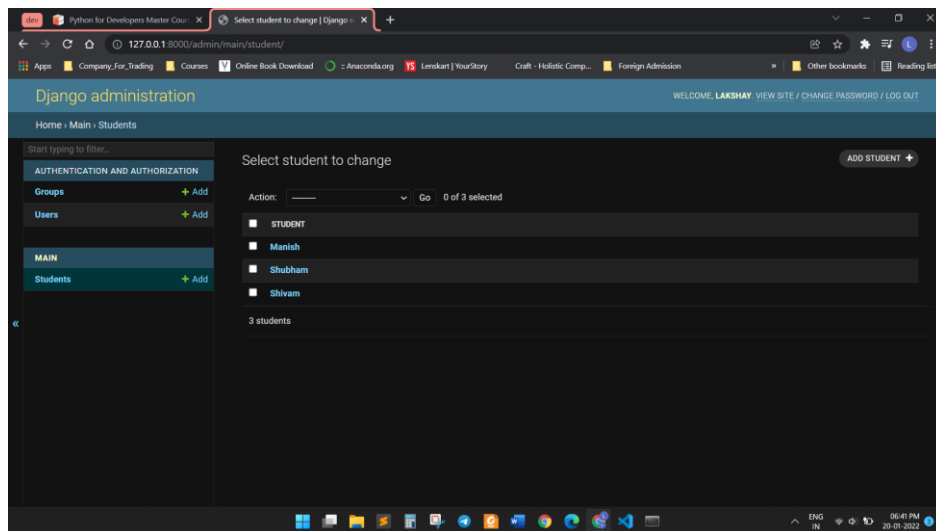
```

(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py makemigrations
Migrations for 'main':
  main\migrations\0002_student_address_student_email_student_gender.py
    - Add field address to student
    - Add field email to student
    - Add field gender to student

(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, main, sessions
Running migrations:
  Applying main.0002_student_address_student_email_student_gender... OK

(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py runserver
Watching for file changes with StatReloader

```



See now it shows the name of student where there are entries.

## Validation in models

We can include the validators in our **models.py** file as:

```
from django.core.validators import (
    EmailValidator,
    MaxValueValidator,
    MinValueValidator,
    URLValidator,
    validate_slug
)
```

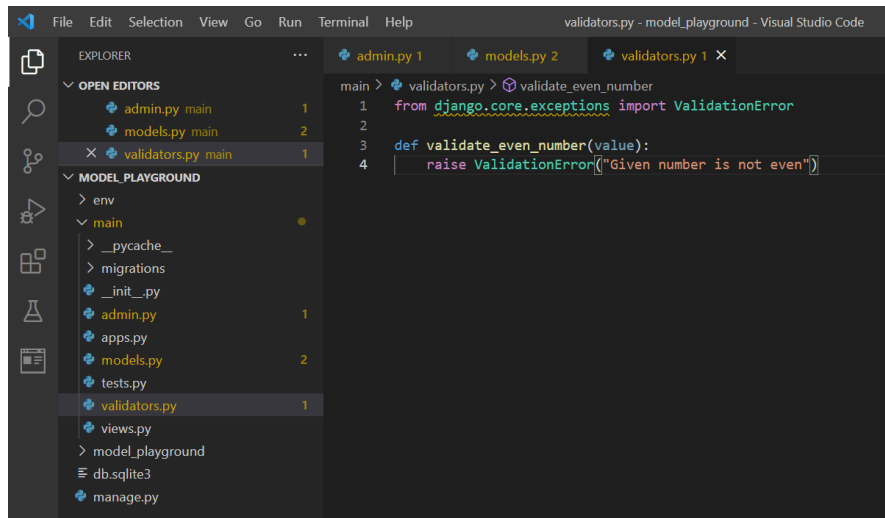
And then can define for the fields of table as:

```
#Define email as the charfield and explicitly providing the validators
email = models.CharField(
    max_length=50, validators=[EmailValidator(message="Please check your email
again")] )

age = models.IntegerField(
    null = True,
    validators= [ MaxValueValidator(150), MinValueValidator(0) ]
)

# slug contain in the url of the website
#They can contain char, numerals, - and _ only
slug = models.CharField(
    max_length=150, validators=[validate_slug], null=True)
```

We can also create our own validators. It is a best practice to make a validators.py file in the main which is our startapp. Now then can import in our models file.



In models.py as

```
from main.validators import (
    validate_even_number # Which is a function define in validator class
)
```

Now user will be able to enter the age only when it is even number.

```
age = models.IntegerField(
    null = True,
    validators= [
        MaxValueValidator(150),
        MinValueValidator(0),
        validate_even_number ]
)
```

Python for Developers Master Course | Add student | Django site admin

127.0.0.1:8000/admin/main/student/add/

Start typing to filter...

**AUTHENTICATION AND AUTHORIZATION**

Groups + Add

Users + Add

**MAIN**

Students + Add

### Add student

Please correct the errors below.

Name: Rohit

Roll no: 2345

Course: B.Tech

Branch: Computer Science

Address: H.No. 584/6, Govind Puri

Email: rohi@gmail.com

Gender: Male

Age: 13

Slug: 89&

Please check your email again

Given number is not even

Enter a valid "slug" consisting of letters, numbers, underscores or hyphens.

Save and add another Save and continue editing SAVE

## Work on creating Relational model in Django

We will create a new app for this, named `relation_playground`.

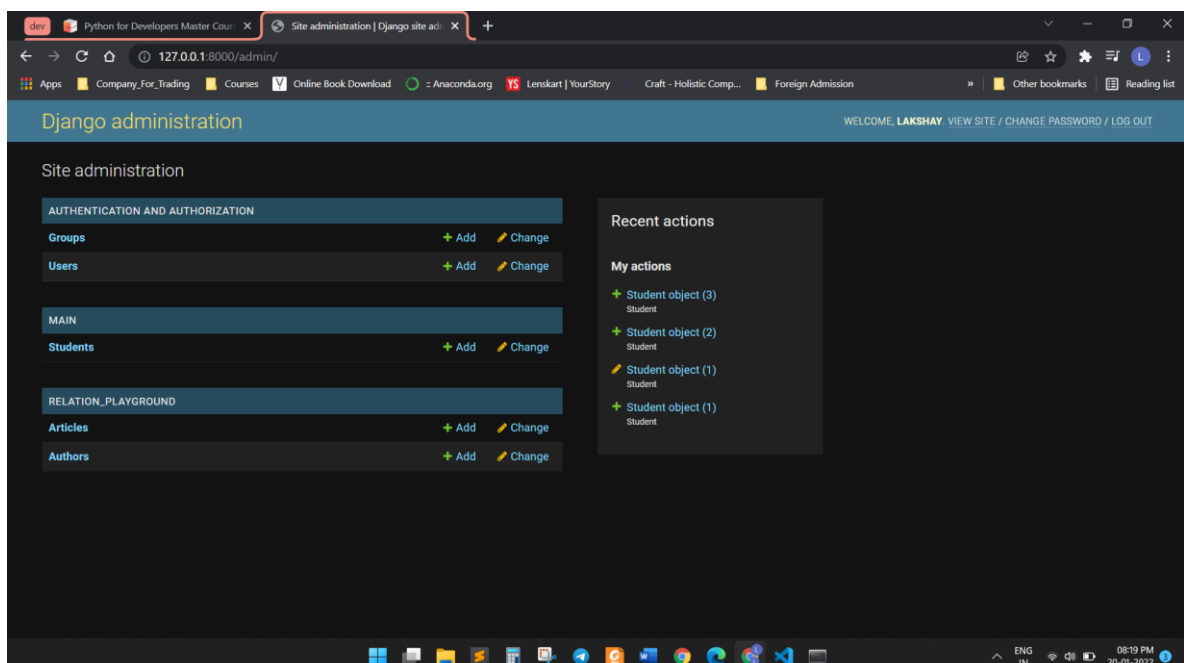
Don't forget to add the name in the **settings.py** file in **INSTALLED\_APPS** variable.

```
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py makemigrations
Migrations for 'relation_playground':
  relation_playground\migrations\0001_initial.py
    - Create model Author
    - Create model Article

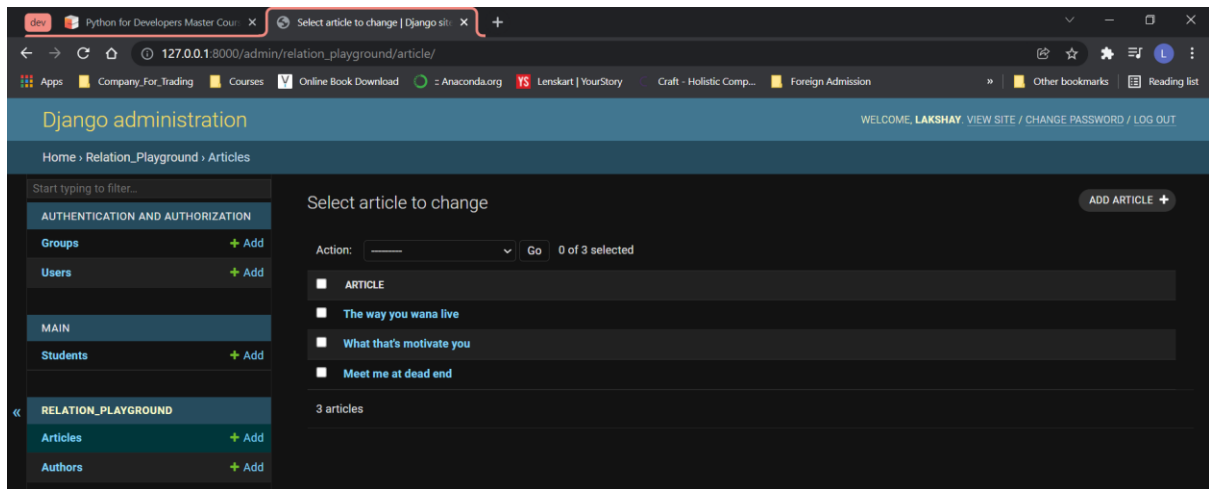
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, main, relation_playground, sessions
Running migrations:
  Applying relation_playground.0001_initial... OK

(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
January 20, 2022 - 20:17:10
Django version 4.0.1, using settings 'model_playground.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```



See now that two added tables Author and Article is also visible on the admin page.



Now we will see some commands in shell for Django:

```
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py shell
Python 3.8.8 (default, Apr 13 2021, 15:08:03) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from relation_playground import models
>>> models.Author.objects
<django.db.models.manager.Manager object at 0x00000147C60A04F0>
>>> models.Author.objects.all()
<QuerySet [<Author: Lakshay Bansal>, <Author: Shubham>, <Author: Jatin>]>
>>> models.Author.objects.get(id = 1)
<Author: Lakshay Bansal>
>>> models.Author.objects.get(name_contains = 'a')
Traceback (most recent call last):
  File "<console>", line 1, in <module>
```

See Author is a class inside models.py which is inside realtion\_playground.

```
>>> from relation_playground import models
>>> author = models.Author.objects.get(id = 1)
>>> author
<Author: Lakshay Bansal>
>>> author.article_set
<django.db.models.fields.related_descriptors.create_reverse_many_to_one_manager.<locals>.RelatedManager object at 0x00000147C60A0730>
>>> author.article_set.all()
<QuerySet [<Article: Meet me at dead end>]>
```

Now we create an object from Author class and store it in a container. And we can find all the article available for that author.

- **Many-To-Many Relation in Tables**

Now we are two more tables to our second app **relation\_playground**.

```
# Many-To-Many relationship
class Topping(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name

class Pizza(models.Model):
    name = models.CharField(max_length=100)
    price = models.IntegerField(validators=[
        MinValueValidator(0)
    ])

    # It will create a many to many mapping with Pizza
    topping = models.ManyToManyField('Topping')
    def __str__(self):
        return self.name
```

```
(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py makemigrations
Migrations for 'relation_playground':
  relation_playground\migrations\0002_topping_pizza.py
    - Create model Topping
    - Create model Pizza

(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, main, relation_playground, sessions
Running migrations:
  Applying relation_playground.0002_topping_pizza... OK
```

```
(env) (base) C:\Users\laksh\Desktop\model_playground>sqlite3 db.sqlite3
SQLite version 3.35.4 2021-04-02 15:20:15
Enter ".help" for usage hints.
sqlite> .tables
auth_group          django_migrations
auth_group_permissions  django_session
auth_permission     main_student
auth_user           relation_playground_article
auth_user_groups    relation_playground_author
auth_user_user_permissions  relation_playground_pizza
django_admin_log    relation_playground_pizza_topping
django_content_type relation_playground_topping
sqlite>
```

Now there is one extra table as we had define **topping = models.ManyToManyField('Topping')** in the models.py file this time.

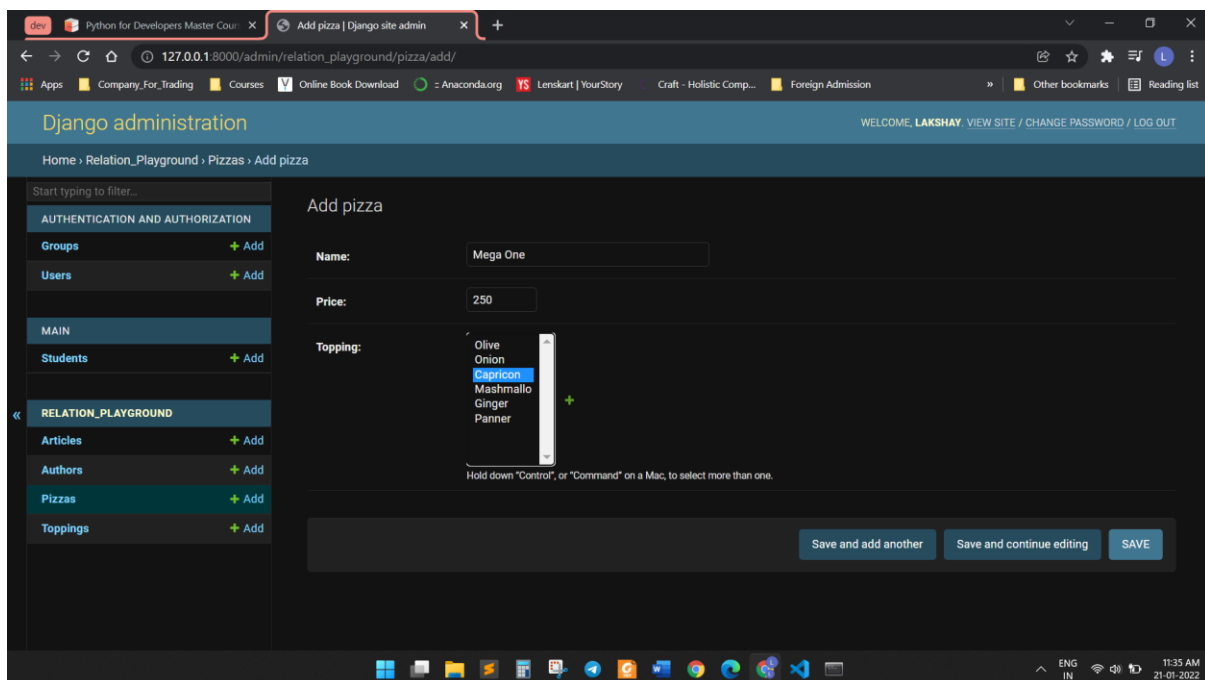
```

sqlite> .schema relation_playground_article
CREATE TABLE IF NOT EXISTS "relation_playground_article" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "title" varchar(250) NOT NULL, "body" text NOT NULL, "author_id" bigint NOT NULL REFERENCES "relation_pl
ayground_author" ("id") DEFERRABLE INITIALLY DEFERRED);
CREATE INDEX "relation_playground_article_author_id_50db7978" ON "relation_playground_article" ("author_id");
sqlite> .schema relation_playground_author
CREATE TABLE IF NOT EXISTS "relation_playground_author" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "name" varchar(50) NOT NULL, "designation" varchar(20) NOT NULL);
sqlite> .schema relation_playground_pizza
CREATE TABLE IF NOT EXISTS "relation_playground_pizza" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "name" varchar(100) NOT NULL, "price" integer NOT NULL);
sqlite> .schema relation_playground_pizza_topping
CREATE TABLE IF NOT EXISTS "relation_playground_pizza_topping" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "pizza_id" bigint NOT NULL REFERENCES "relation_playground_pizza" ("id") DEFERRABLE INITIALLY DEFE
RRED, "topping_id" bigint NOT NULL REFERENCES "relation_playground_topping" ("id") DEFERRABLE INITIALLY DEFERRED);
CREATE UNIQUE INDEX "relation_playground_pizza_topping_pizza_id_topping_id_b269e09d_uniq" ON "relation_playground_pizza_topping" ("pizza_id", "topping_id");
CREATE INDEX "relation_playground_pizza_topping_pizza_id_a2092685" ON "relation_playground_pizza_topping" ("pizza_id");
CREATE INDEX "relation_playground_pizza_topping_topping_id_d1840481" ON "relation_playground_pizza_topping" ("topping_id");
sqlite>

```

As Article table there is author\_id to act as a foreign key for Author and Article. But such bridge is not present in the Pizza and Topping table instead there is a many to many mapping table is created which is a table **relation\_playground\_pizza\_topping**.

Now from front end we added some types of topping in the toppings table. Then in Pizza we can select the topping as:



We can extract the information provided from front through backend using Django shell.

```

(env) (base) C:\Users\laksh\Desktop\model_playground>python manage.py shell
Python 3.8.8 (default, Apr 13 2021, 15:08:03) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from relation_playground import models
>>> pzObj = models.Pizza.objects.get(id=2)
>>> pzObj
<Pizza: King Size>
>>> pzObj.topping
<django.db.models.fields.related_descriptors.create_forward_many_to_many_manager.<locals>.ManyRelatedManager object at 0x000001E8C0D5A8B0>
>>> pzObj.topping.all()
<QuerySet [<Topping: Olive>, <Topping: Onion>, <Topping: Panner>]>

```

As it is a many to many mapping. We can also get the orders that are placed for particular topping.

```
>>> tObj = models.Topping.objects.get(id=3)
>>> tObj.pizza_set.all()
<QuerySet [<Pizza: Mega One>]>
>>> tObj
<Topping: Capricon>
```